



SYNTHÈSE

Programmation procédurale :

Programmation structurée & modulaire

INTRODUCTION

Les machines programmables (ordinateurs, automates, tablettes...) n'ont besoin que de **4 groupes d'instructions** de la programmation impérative pour pouvoir exécuter l'ensemble des programmes imaginables.

Ces instructions sont pour rappel :

1. Les instructions d'entrée & de sortie (ou écriture & lecture)
2. Les instructions d'affectations (déclaration et affectation des variables)
3. Les instructions de la structure conditionnelle
4. Les instructions de la structure itérative

INTRODUCTION

Cependant afin de faciliter le développement de programmes, d'autres paradigmes peuvent s'ajouter ou supplanter le paradigme impératif.

En effet la programmation impérative est tout à fait satisfaisant pour développer des programmes de taille modeste. Mais le développement de programme de plus grande envergure et notamment dans le cadre d'un code développé en équipe, il est intéressant de mettre en œuvre la programmation procédurale.

INTRODUCTION

Les apports du paradigme de la programmation procédurale sont les suivants :

- Renforcement de la **structuration** du code (factorisation du code redondant notamment)
- **Modularisation** du code :
 - Possibilité de scinder le code en plusieurs fichiers facilitant le travail en équipe
 - Possibilité de réutiliser le code dans d'autres programmes (utilisation de modules)

EXEMPLE : PROGRAMME DE CONVERSION HORAIRE DÉCIMAL

Interface avec l'utilisateur :

```
>>> %Run 'Exemple Cours.py'
```

```
Saisissez votre durée sous le format HH:MM:SS = 12:05:32
```

```
Vous avez saisi la durée suivante : 12 h  5 min 32 s
```

```
La durée exprimée en seconde :          43532 s
```

```
La durée exprimée en minute décimale :   725.5333333333333 min
```

```
La durée exprimée en heure décimale :    12.092222222222222 h
```

EXEMPLE : PROGRAMME DE CONVERSION HORAIRE DÉCIMAL

Code du programme :

Le code est composé de 2 parties :

1. La définition des fonctions du programme
2. La partie principale du programme contenant l'appel des fonctions

1. Définition des fonctions

```
1 # DEFINITION DES FONCTIONS DU PROGRAMME :
2 # -----
3
4 def extraction(hhmmss):
5     h = int(hhmmss[0] + hhmmss[1])
6     m = int(hhmmss[3] + hhmmss[4])
7     s = int(hhmmss[6] + hhmmss[7])
8     return h, m, s
9
10 def convSecondeDeci(hhmmss):
11     h, m, s = extraction(hhmmss)
12     return h * 3600 + m * 60 + s
13
14 def convMinuteDeci(hhmmss):
15     h, m, s = extraction(hhmmss)
16     return h * 60 + m + s / 60
17
18 def convHeureDeci(hhmmss):
19     h, m, s = extraction(hhmmss)
20     return h + m / 60 + s / 3600
21
22
23 # PROGRAMME PRINCIPAL :
24 # -----
25
26 heureTexte = input("Saisissez votre durée sous le format HH:MM:SS = ")
27 h, m, s = extraction(heureTexte)
28 print("\nVous avez saisi la durée suivante :", h, "h ", m, "min", s, "s")
29 print("\n\tLa durée exprimée en seconde :\t\t", convSecondeDeci(heureTexte), "s")
30 print("\tLa durée exprimée en minute décimale :\t", convMinuteDeci(heureTexte), "min")
31 print("\tLa durée exprimée en heure décimale :\t", convHeureDeci(heureTexte), "h")
```

2. Appel des fonctions

EXEMPLE : PROGRAMME DE CONVERSION HORAIRE DÉCIMAL

Code du programme :

La partie principale du programme contenant l'appel des fonctions

```
22
23 # PROGRAMME PRINCIPAL :
24 # -----
25
26 heureTexte = input("Saisissez votre durée sous le format HH:MM:SS = ")
27 h, m, s = extraction(heureTexte)
28 print("\nVous avez saisi la durée suivante :", h, "h ", m, "min", s,"s")
29 print("\n\tLa durée exprimée en seconde :\t\t", convSecondeDeci(heureTexte), "s")
30 print("\tLa durée exprimée en minute décimale :\t", convMinuteDeci(heureTexte), "min")
31 print("\tLa durée exprimée en heure décimale :\t", convHeureDeci(heureTexte), "h")
```

EXEMPLE : PROGRAMME DE CONVERSION HORAIRE DÉCIMAL

Code du programme :

La définition des fonctions du programme

```
1  # DEFINITION DES FONCTIONS DU PROGRAMME :
2  # -----
3
4  def extraction(hhmmss):
5      h = int(hhmmss[0] + hhmmss[1])
6      m = int(hhmmss[3] + hhmmss[4])
7      s = int(hhmmss[6] + hhmmss[7])
8      return h, m ,s
9
10 def convSecondeDeci(hhmmss):
11     h, m, s = extraction(hhmmss)
12     return h * 3600 + m * 60 + s
13
14 def convMinuteDeci(hhmmss):
15     h, m ,s = extraction(hhmmss)
16     return h * 60 + m + s / 60
17
18 def convHeureDeci(hhmmss):
19     h, m ,s = extraction(hhmmss)
20     return h + m /60 + s / 3600
21
```


DÉFINITIONS & CODAGE

DÉFINITIONS FONDAMENTALES :

Procédure : Partie de code d'un programme qui peut être exécuté par une instruction d'appel. La procédure pourra prendre un ou plusieurs paramètres pour qu'elle puisse travailler avec les données pertinentes.

Fonction : Une fonction tout comme une procédure est une partie de code isolée pouvant être appelée avec des paramètres de travail par le programme principal ou une autre fonction/procédure. Par contre, à la différence d'une procédure, une fonction retournera une ou plusieurs valeurs.

REMARQUE : En pratique on ne fera pas la distinction entre procédure et fonction. On utilisera le terme fonction pour désigner l'un ou l'autre.

FORME CANONIQUE DU CODE

Définition de la fonction :

Paramètres/arguments formels de la fonction

Entête de la fonction

Corps de la fonction

```
def IdFonction(arg01, arg02,...):  
    #Code associé au bloc d'instructions  
    return retour01, retour02, ...
```

Le retour n'est pas obligatoire (distinction fonction et procédure)



Le définition d'une fonction doit toujours précéder son appel.

FORME CANONIQUE DU CODE

Appel de la fonction :

Paramètres/arguments effectifs de la fonction

```
IdFonction(arg01, arg02, ...)
```



Le définition d'une fonction doit toujours précéder son appel.

FORME CANONIQUE DU CODE

Cas des fonctions avec plusieurs valeurs de retours

Solution 1 : Utiliser un identifiant qui sera un tuple contenant l'ensemble des valeurs

```
1 def foncTest():  
2     return "rouge", "bleu", 33  
3  
4 resultat = foncTest()  
5 print("resultat est du type : " + str(type(resultat)))  
6 print("Contenu de resultat : " + str(resultat))  
7
```

```
Console x  
  
>>> %Run TEMP.py  
resultat est du type : <class 'tuple'>  
Contenu de resultat : ('rouge', 'bleu', 33)  
  
>>>
```

FORME CANONIQUE DU CODE

Cas des fonctions avec plusieurs valeurs de retours

Solution 2 : Accueillir les 'n' valeurs de retour dans 'n' variables de réception.

```
1 def foncTest():  
2     return "rouge", "bleu", 33  
3  
4 a, b, c = foncTest()  
5  
6 print(a, ", ", b, ", ", c)  
7
```

Console x

Contenu de resultat : ('rouge', 'bleu', 33)

>>> %Run TEMP.py

rouge , bleu , 33

>>>

APPLICATION

Réalisez la 1^{ère} série de TP d'application

PORTÉE & DURÉE DE VIE DES VARIABLES LOCALES D'UNE FONCTION

PORTÉES DES VARIABLES

La portée d'une variable dépend de l'espace de nom qui le contient.

Espace des noms global :

Espace des noms local de la fonction :

```
def fonction01(...):  
    ...  
    return ...
```

Espace des noms local de la fonction :

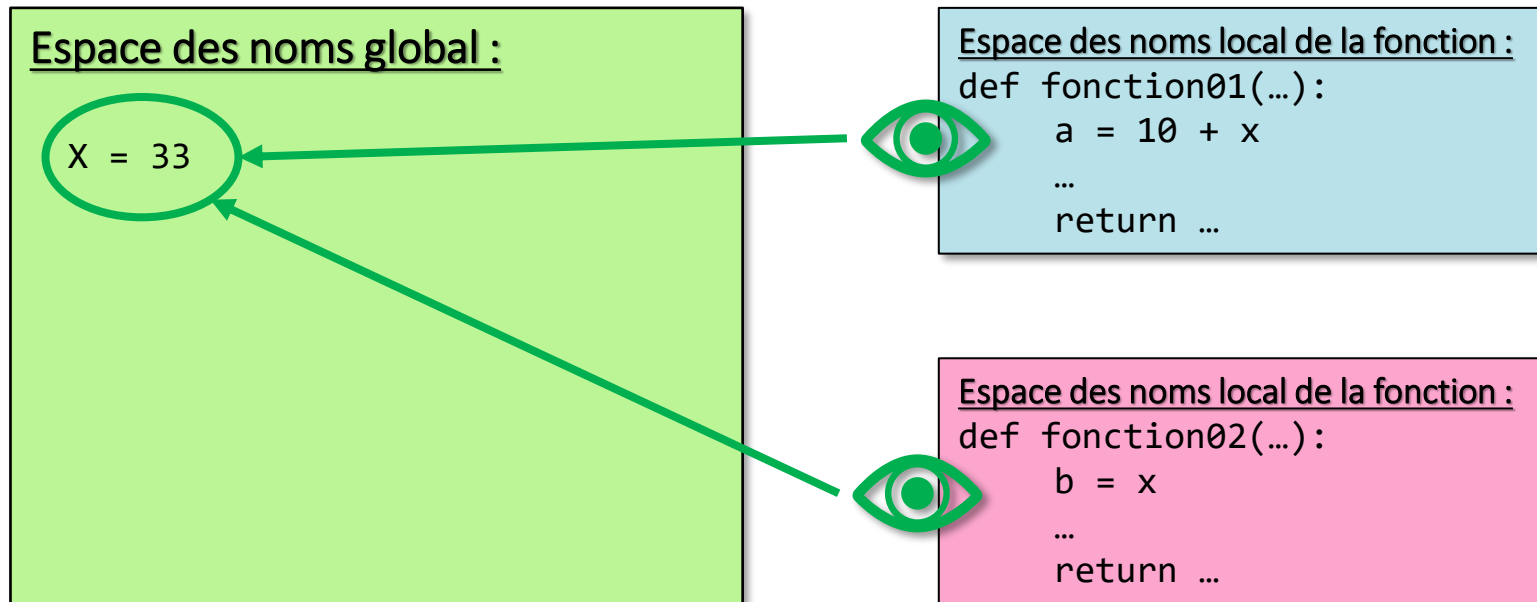
```
def fonction02(...):  
    ...  
    return ...
```

Code du programme principal...

...

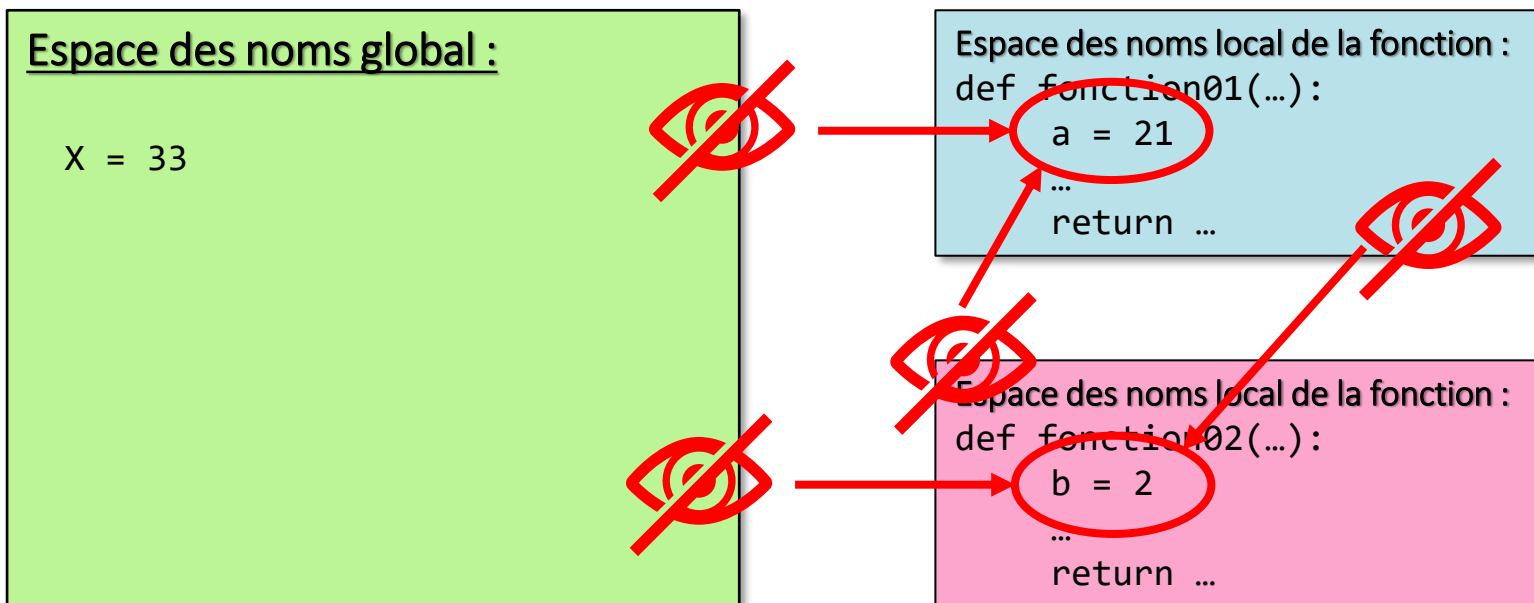
PORTÉES DES VARIABLES

Une variable contenue dans l'espace des noms global est visible dans tous les espaces locaux. On peut donc récupérer sa valeur. On dit que x est une variable globale



PORTÉES DES VARIABLES

La réciproque est totalement fausse, la variable **a** de **fonction01** n'est ni visible du programme principal, ni des autres fonctions.

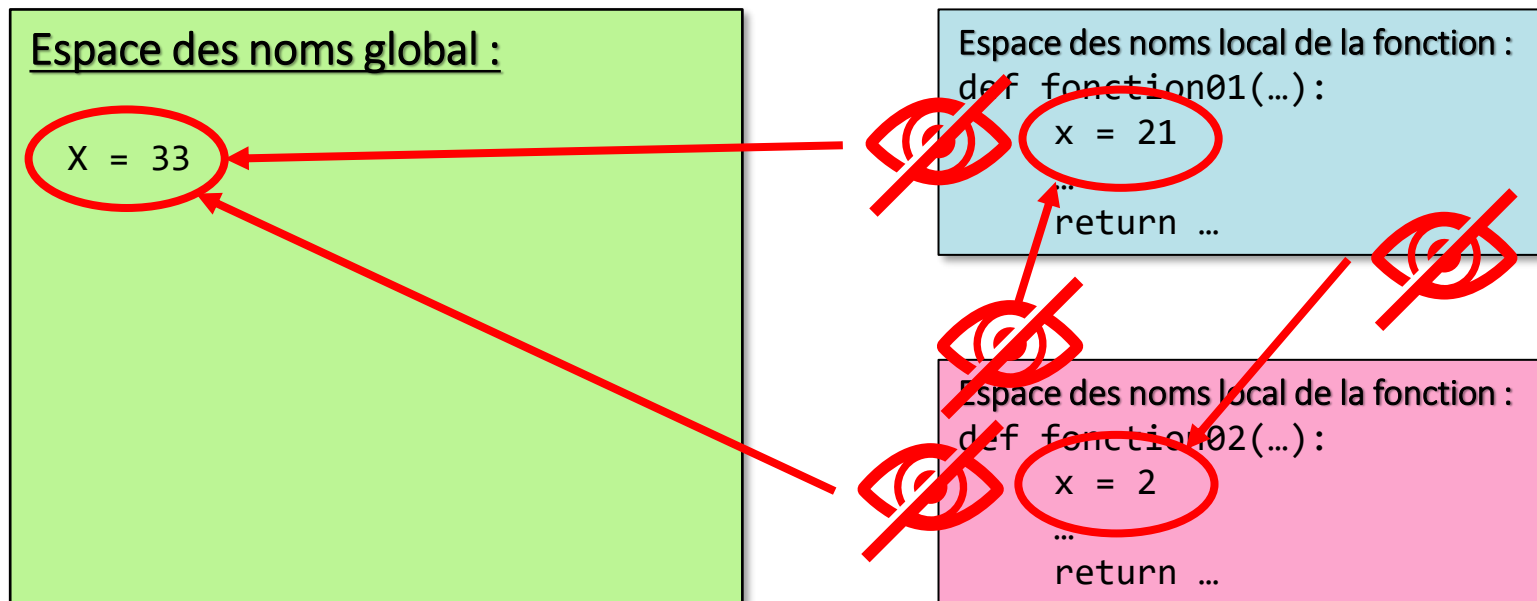


Nom des variables par rapport à leur espace :

- `global.x = 33`
- `fonction01.a = 21`
- `fonction02.b = 2`

PORTÉES DES VARIABLES

Quand des variables portent le même identifiant, les variables de l'espace local vont rendre inaccessible les variables globales portant le même identifiant.

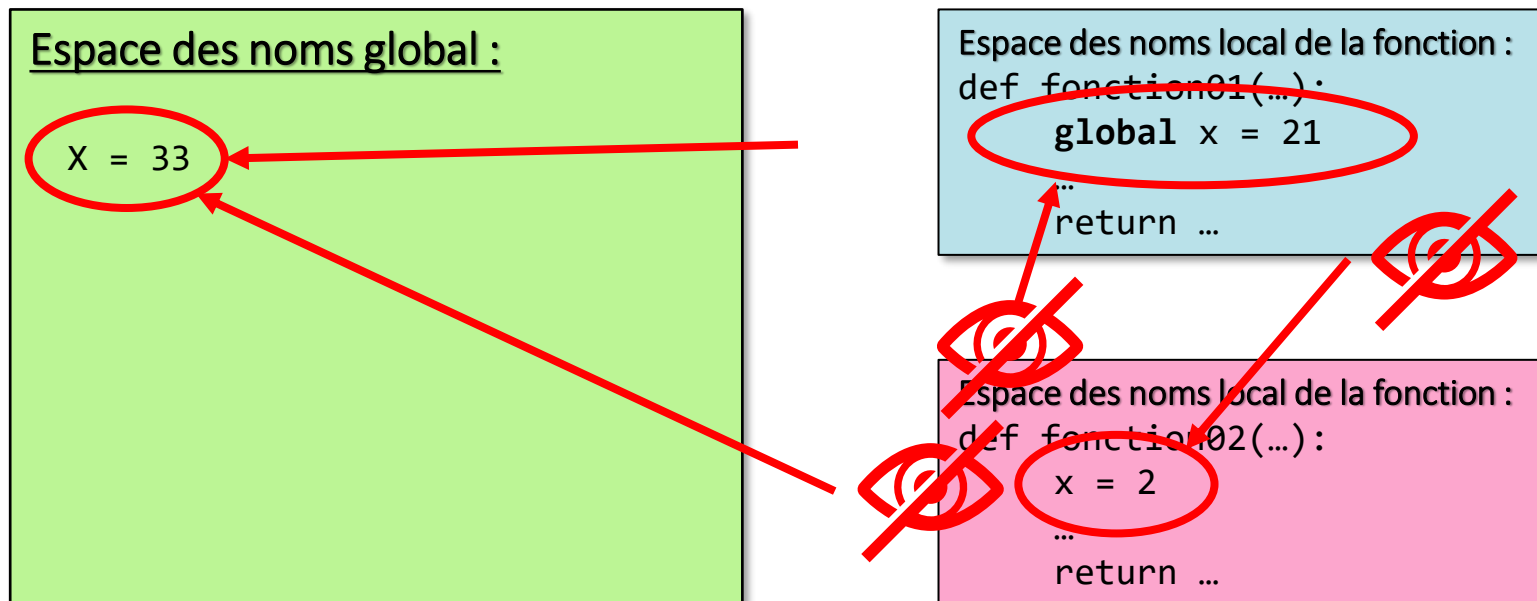


Nom des variables par rapport à leur espace :

- `global.x = 33`
- `fonction01.x = 21`
- `fonction02.x = 2`

PORTÉES DES VARIABLES

On peut **forcer la modification** d'une variable globale en utilisant le mot clé **"global"** en déclarant la variable dans la fonction



Nom des variables par rapport à leur espace :

- `global.x = 33`
- `fonction02.x = 2`

PORTÉES DES VARIABLES

Exemple :

```
1 varGlobale = 33
2
3 def test():
4     return varGlobale + 7
5
6 print(test())
7
```

```
Console x
rouge , bleu , 33
>>> %Run TEMP.py
40
>>>
```

PORTÉES DES VARIABLES

Exemple :

```
1 varGlobale = 33
2
3 def test():
4     varGlobale = varGlobale + 7 # Erreur ! on ne peut pas modifier une variable globale
5     return varGlobale
6
7 print(test())
8
```

```
>>> %Run TEMP.py
Traceback (most recent call last):
  File "D:\FichiersUtilisateurs\ljules\Desktop\TEMP.py", line 7, in <module>
    print(test())
  File "D:\FichiersUtilisateurs\ljules\Desktop\TEMP.py", line 4, in test
    varGlobale = varGlobale + 7 # Erreur ! on ne peut pas modifier une variable globale
UnboundLocalError: local variable 'varGlobale' referenced before assignment

>>>
```

PORTÉES DES VARIABLES

Exemple :

```
1 varGlobale = 33
2
3 def test():
4     varGlobale = 7 # Cette variable locale est différent de la variable globale de même identifiant !!!
5     return varGlobale
6
7 print(test())
8
```

```
>>> %Run TEMP.py
7
>>>
```


PORTÉES DES VARIABLES

Exemple :

```
1 def test():  
2     global varGlobale  
3     varGlobale = 10  
4  
5 test()  
6 print(varGlobale)  
7  
8
```

```
>>> %Run TEMP.py  
10  
>>>
```

PORTÉES DES VARIABLES

A retenir :

Durant l'exécution d'une fonction, le système va vérifier pour chaque variable présente dans la fonction si elle a un caractère **local** ou **global**, c'est le concept d'espaces des noms des variables.

Si la variable n'a pas été déclarée dans le corps de la fonction (espace local), le système va alors vérifier si la déclaration n'est pas présente dans le programme principal (espace global).

Une variable globale est consultable mais ne peut être modifiée par la fonction, sauf si la fonction la déclare avec l'attribut "global".

DURÉE DE VIE D'UNE VARIABLE LOCALE

Une fonction et ses variables locales existent au moment de son appel et jusqu'à l'exécution de la dernière instruction du corps de la fonction.

Certains langages permettent de conserver les valeurs des variables entre 2 appels d'une fonction.

Dans le cas de Python il n'y a pas de possibilité de conserver l'état des variables. En effet toutes les variables locales sontinstanciées à l'appel de la fonction et sont détruites à la fin de l'exécution de la fonction.

ARGUMENTS PAR DÉFAUT

SYNTAXE

En Python il est possible d'attribuer des valeurs par défaut aux arguments.

Sur l'appel de la fonction, on peut utiliser le nom des arguments pour lesquels on désire modifier les valeurs par défaut

```
1 def fonc(coul01 = "rouge", coul02 = "vert", coul03 = "jaune"):  
2     print(coul01, coul02, coul03)  
3  
4 fonc()  
5 fonc("bleu", "marron", "violet")  
6 fonc(coul02 = "bleu")
```

```
>>> %Run TEMP.py  
rouge vert jaune  
bleu marron violet  
rouge bleu jaune  
>>>
```

Fin